

Dirichlet-Q AlphaZero Algorithms

Notation

B = number of root states searched in parallel,
 A = maximum number of actions,
 K = number of Thompson samples per root state per round,
 R = number of posterior update rounds,
 H = internal rollout horizon after the root action,
 M = number of Monte Carlo samples for the policy target,
 $s \in \mathcal{S}^B$,
 $\alpha \in \mathbb{R}_{>0}^{B \times A \times 3}$,
 $a \in \mathbb{N}^{B \times K}$,
 $d \in \Delta^2$,
 $\lambda \geq 0$.

The utility used for search selection and policy improvement is

$$U(\phi) = \phi_W - \phi_L.$$

Generic Batched Thompson Root Search

Algorithm 1: Dirichlet Q-head parameters

Input: state s , action a , network parameters θ

$$\bar{\phi}_\theta^Q(s, a) \leftarrow \text{softmax}(r_\theta^Q(s, a))$$

$$\alpha_{\theta,0}^Q(s, a) \leftarrow \exp(t_\theta^Q(s, a))$$

$$\alpha_\theta^Q(s, a) \leftarrow \alpha_{\theta,0}^Q(s, a) \bar{\phi}_\theta^Q(s, a)$$

return $\alpha_\theta^Q(s, a)$

Algorithm 2: ThompsonSampleActions(α, K, legal)

Input: $\alpha \in \mathbb{R}_{>0}^{B \times A \times 3}$, K , $\text{legal} \in \{0, 1\}^{B \times A}$

Sample $\phi \sim \text{Dirichlet}(\alpha)$ with $\phi \in \mathbb{R}^{B \times K \times A \times 3}$

$$u \leftarrow U(\phi)$$

$$u \leftarrow \text{MaskIllegal}(u, \text{legal}, -\infty)$$

$$a \leftarrow \arg \max_{\text{action}} u$$

return a

Algorithm 3: GetEvidence(s_f, θ)

Input: final rollout state s_f , network parameters θ
if s_f is terminal with outcome z **then**
 $d \leftarrow e_z$
 $\lambda \leftarrow c_{\text{terminal}}$
else
 $d \leftarrow \text{softmax}(r_\theta^V(s_f))$
 $\lambda \leftarrow 0$
end if
 $d \leftarrow \text{flip_to_root_perspective}(d)$
return (d, λ)

Algorithm 4: InternalRollout(s, a, θ, H)

Input: root state batch s , root actions a , network parameters θ , horizon H
 $s' \leftarrow \text{Step}(s, a)$
for $h = 1, \dots, H$ **do**
 $\pi \leftarrow \text{softmax}(\ell_\theta(s', \cdot))$
 $a' \leftarrow \text{PolicyAction}(\pi)$
 $s' \leftarrow \text{Step}(s', a')$
end for
 $(d, \lambda) \leftarrow \text{GetEvidence}(s', \theta)$
return (d, λ)

Algorithm 5: UpdateAlphas(α, a, d, λ)

Input: current posterior α , sampled actions a , rollout evidence (d, λ)
 $\Delta\alpha_{b,k} \leftarrow \lambda_{b,k} d_{b,k}$
for each batch element b and action j **do**
 $\alpha_{b,j}^{\text{new}} \leftarrow \alpha_{b,j} + \sum_{k: a_{b,k}=j} \Delta\alpha_{b,k}$
end for
return α^{new}

Algorithm 6: BatchedThompsonRootSearch(s, θ, R, K, H)

Input: root state batch s , network parameters θ , rounds R , samples K , horizon H
 $\alpha \leftarrow \alpha_\theta^Q(s, \cdot)$
 $\text{legal} \leftarrow \text{LegalActions}(s)$
for $r = 1, \dots, R$ **do**
 $a \leftarrow \text{ThompsonSampleActions}(\alpha, K, \text{legal})$
 $(d, \lambda) \leftarrow \text{InternalRollout}(s, a, \theta, H)$
 $\alpha \leftarrow \text{UpdateAlphas}(\alpha, a, d, \lambda)$
end for
 $\alpha_{\text{search}}(s, \cdot) \leftarrow \alpha$
return $\alpha_{\text{search}}(s, \cdot)$

Search-Improved Policy Target

Algorithm 7: PosteriorBestPolicyTarget($\alpha_{\text{search}}, M, \text{legal}$)

Input: search posterior $\alpha_{\text{search}} \in \mathbb{R}_{>0}^{B \times A \times 3}$, samples M , legal mask legal

Sample $\phi_b^{(m)} \sim \text{Dirichlet}(\alpha_{\text{search}}(b, \cdot))$ for $m = 1, \dots, M$

$u_{b,a}^{(m)} \leftarrow U(\phi_{b,a}^{(m)})$

$u_{b,a}^{(m)} \leftarrow \text{MaskIllegal}(u_{b,a}^{(m)}, \text{legal}, -\infty)$

$a_{b,m}^* \leftarrow \arg \max_a u_{b,a}^{(m)}$

$\hat{\pi}_{\text{search}}(a \mid s_b) \leftarrow \frac{1}{M} \sum_{m=1}^M \mathbf{1}[a_{b,m}^* = a]$

return $\hat{\pi}_{\text{search}}(\cdot \mid s)$

Equivalently,

$$\pi_{\text{search}}(a \mid s) = \mathbb{P} \left(a = \arg \max_b U(\phi_b) \right), \quad \phi_b \sim \text{Dirichlet}(\alpha_{\text{search}}(s, b)).$$

Planned Terminal-Only MCTX Implementation

Algorithms 8 and 9 are not a second search rule. They describe how `play.py` should realize the root-search primitive inside the fixed-size MCTX tree machinery. MCTX owns the simulation loop and tree storage. The Dirichlet-Q logic owns the root action selection rule and the terminal-evidence posterior updates performed between MCTX blocks.

For a search budget T , MCTX allocates

$$N = T + 1$$

nodes per root state. Node 0 is the root, and each simulation can expand at most one additional node. The tree contains array fields such as

$$\begin{aligned} \text{node_visits} &\in \mathbb{N}^{B \times N}, \\ \text{parents} &\in \mathbb{N}^{B \times N}, \\ \text{action_from_parent} &\in \mathbb{N}^{B \times N}, \\ \text{children_index} &\in \mathbb{N}^{B \times N \times A}, \\ \text{children_prior_logits} &\in \mathbb{R}^{B \times N \times A}, \\ \text{children_visits} &\in \mathbb{N}^{B \times N \times A}. \end{aligned}$$

In this implementation, the important custom field is the per-node embedding. For each tree node n , `play.py` stores

$$\text{embedding}_n = \left(s_n, d_n, w_n, a_n^{\text{root}}, p_n, \alpha_\theta^Q(s_n, \cdot) \right),$$

where s_n is the environment state at the node, d_n is the WDL distribution in the node-local player-to-move perspective, w_n is the Dirichlet evidence weight, a_n^{root} is the first action taken from the root on the path to n , and $p_n \in \{0, 1\}$ is the depth parity relative to the root.

When MCTX expands a child by action a from a parent node, `play.py` applies the environment step and evaluates the resulting observation:

$$(s', \ell_\theta(s', \cdot), \alpha_\theta^V(s'), \alpha_\theta^Q(s', \cdot)).$$

If s' is terminal, the stored WDL distribution is terminal one-hot evidence in the child-local perspective and $w_n = c_{\text{terminal}}$. Otherwise, the stored WDL distribution may be the value-head mean

$$d_n = \bar{\phi}_\theta^V(s')$$

for MCTX’s scalar value backup, but it is not Dirichlet-Q evidence and $w_n = 0$. The root action is set to the action taken from the root at depth one and then inherited by all deeper descendants. The depth parity flips at every ply.

The terminal root evidence represented by the tree is

$$\text{RootEvidenceSum}(\text{tree})_{b,a} = \sum_{n=1}^{N-1} \mathbf{1} \left[\begin{array}{l} a_{b,n}^{\text{root}} = a, \\ \text{node_visits}_{b,n} > 0, \\ s_{b,n} \text{ is terminal} \end{array} \right] w_{b,n} \text{align}_{\text{root}}(d_{b,n}, p_{b,n}).$$

where $\text{align}_{\text{root}}(d, p)$ flips the WDL vector when $p = 1$ and leaves it unchanged when $p = 0$. Thus only terminal evidence is accumulated in the root player’s perspective and bucketed by the first action chosen from the root.

Each MCTX simulation proceeds as follows:

select a path from the root,
expand one edge with the recurrent function,
store node evidence,
run MCTX’s visit/value backup.

The three MCTX subroutines in Algorithm 9 have the following roles.

- **MCTXSimulate** starts at the root node. At depth 0 it calls **MCTXRootThompsonSelect**. At deeper nodes it calls the configured interior selector. It follows already-expanded child pointers `children_index` until it finds an unvisited edge, or until the maximum depth is reached. It returns the parent node index i and action a to evaluate next. This step only reads the tree; it does not call the environment or the network.
- **MCTXExpandAndStoreEvidence** evaluates the selected edge (i, a) . It reads the parent embedding, applies $\text{Step}(s_i, a)$, evaluates the resulting state with the network, and writes a child node into the tree. The stored child embedding contains the new environment state, terminal WDL evidence d_n when the node is terminal, evidence weight w_n equal to c_{terminal} for terminal nodes and 0 otherwise, the inherited root action a_n^{root} , the flipped depth parity p_n , and the local Q prior $\alpha_\theta^Q(s_n, \cdot)$. Non-terminal value predictions may still be stored for scalar backup, but they do not update Dirichlet-Q alphas. This step also records the tree topology: parent pointer, action-from-parent, child index, prior logits, reward, and discount.
- **MCTXBackward** runs MCTX’s ordinary scalar backup from the expanded child back to the root. Along the path it increments node and edge visit counts and updates averaged scalar values using the recurrent reward and discount. These scalar values are useful for MCTX bookkeeping and interior selection, but the Dirichlet-Q root posterior is updated only from terminal WDL evidence through $\text{RootEvidenceSum}(\text{tree})$.

The final policy target does not use MCTX visit counts directly. After an MCTX block finishes, it uses the Dirichlet posterior

$$\alpha_{\text{search}} = \alpha_\theta^Q(s, \cdot) + \text{RootEvidenceSum}(\text{tree})$$

and then applies Algorithm 7. Within one call to Algorithm 9,

$$K = 1.$$

Each pass through the loop is one Thompson root selection from the block’s starting posterior α_{base} . Terminal Dirichlet evidence found inside the block is summed after the loop, not used to mutate alphas inside the loop.

Algorithm 8: MCTXRootThompsonSelect(tree)

```

 $\alpha_{\text{base}} \leftarrow \text{tree.extra\_data.alpha\_Q\_prior}$ 
Sample  $\phi \sim \text{Dirichlet}(\alpha_{\text{base}})$ 
 $u \leftarrow U(\phi)$ 
 $u \leftarrow \text{MaskIllegal}(u, \text{tree.root\_invalid\_actions}, -\infty)$ 
 $a \leftarrow \arg \max_{\text{action}} u$ 
return  $a$ 

```

Algorithm 9: MCTXDirichletRootSearch($s, \theta, T, \alpha_{\text{base}}$)

```

Input: root state  $s$ , network parameters  $\theta$ , simulations  $T$ , starting posterior  $\alpha_{\text{base}}$ 
if  $\alpha_{\text{base}}$  is omitted then  $\alpha_{\text{base}} \leftarrow \alpha_{\theta}^Q(s, \cdot)$  end if
tree  $\leftarrow \text{InstantiateTreeFromRoot}(s, \theta, N = T + 1, \alpha_{\text{base}})$ 
for  $t = 1, \dots, T$  do
    Root Thompson selection uses  $\alpha_{\text{base}}$ 
     $(i, a) \leftarrow \text{MCTXSimulate}(\text{tree}, \text{root\_selection} = \text{MCTXRootThompsonSelect})$ 
    tree  $\leftarrow \text{MCTXExpandAndStoreEvidence}(\text{tree}, i, a, \theta)$ 
    tree  $\leftarrow \text{MCTXBackward}(\text{tree})$ 
end for
 $E_T \leftarrow \text{RootEvidenceSum}(\text{tree})$ 
 $\alpha_{\text{search}} \leftarrow \alpha_{\text{base}} + E_T$ 
return (tree,  $E_T$ ,  $\alpha_{\text{search}}$ )

```

Algorithm 10: RepeatedMCTXDirichletRootSearch(s, θ, G, T)

```

Input: root state  $s$ , network parameters  $\theta$ , outer rounds  $G$ , simulations per round  $T$ 
 $\alpha^{(0)} \leftarrow \alpha_{\theta}^Q(s, \cdot)$ 
 $E_{\text{total}} \leftarrow 0$ 
for  $g = 1, \dots, G$  do
     $(\text{tree}_g, E_g, \alpha^{(g)}) \leftarrow \text{MCTXDirichletRootSearch}(s, \theta, T, \alpha^{(g-1)})$ 
     $E_{\text{total}} \leftarrow E_{\text{total}} + E_g$ 
end for
 $\alpha_{\text{search}} \leftarrow \alpha^{(G)} = \alpha_{\theta}^Q(s, \cdot) + E_{\text{total}}$ 
return ( $\{\text{tree}_g\}_{g=1}^G, E_{\text{total}}, \alpha_{\text{search}}$ )

```

For the planned implementation,

$$T = \text{num_simulations}, \quad M = \text{policy_mc_samples},$$

and the current `play.py` search shape is closest to $G = 1$. Algorithm 10 is the planned outer wrapper for repeated MCTX blocks; it performs G calls to Algorithm 9 and carries the posterior $\alpha^{(g)}$ into the next call.